SIH26127 · CITY-WIDE AI TRAFFIC INTELLIGENCE ENGINE

# Run & Deployment Guide

Everything needed to clone this repository and get the full stack — backend, AI pipeline, and frontend — running locally and live on a real server.

✓ CI passing on GitHub Actions

✓ 15/15 build phases complete

✓ Real PostgreSQL + PostGIS tested

**Repository:** github.com/maddy-bit/SIH_Project

**Default branch:** main

**Stack:** Spring Boot (Java 21) · Python AI service (YOLOv8, EasyOCR, ByteTrack) · Next.js frontend

# Contents


1. What you're deploying 2

2. Prerequisites 3

3. Clone the repository 4

4. Run it locally — all three services 4

5. Load the demo data 5

6. Verify everything works 6

7. Switch to the production database (PostgreSQL + PostGIS) 7

8. Run the whole stack with Docker Compose 7

9. Go live — deploying to a real server 8

10. Security checklist before going live 10

11. API reference 11

12. Troubleshooting 12

13. What's production-ready vs. not 12


# 1. What you're deploying

This project is a working prototype for **SIH26127**: a city-wide AI engine for multi-camera ANPR (automatic number-plate recognition), cross-camera vehicle trajectory reconstruction, a live map investigation dashboard, and city traffic analytics — built on real pretrained AI models running against demo CCTV-style footage.



```
    ▼
PostgreSQL + PostGIS  (or H2 for zero-config local dev)
```

Three independent services, three languages, one product:

| Service | Tech | Default port | Role |
|---|---|---|---|
| Backend | Spring Boot (Java 21) | 8080 | REST API, database, trajectory-scoring engine, WebSocket live feed, auth, audit log |
| AI service | Python (FastAPI, YOLOv8, EasyOCR) | 8000 | Vehicle detection, tracking, plate OCR — called by scripts, not the browser |
| Frontend | Next.js (React, TypeScript, Tailwind) | 3000 | Investigation dashboard, map, analytics UI |

> **Status**
>
> All 15 build phases are complete and the project has been pushed further into real production-hardening: a genuine PostgreSQL + PostGIS datasource (tested end-to-end, with a working spatial query endpoint), HTTP Basic auth on write/audit endpoints, audit logging of every search, Docker packaging, and a CI pipeline that is confirmed green on GitHub Actions.

## 2. Prerequisites

| Tool | Version | Check with |
|---|---|---|
| Java (JDK) | 21+ | `java -version` |
| Maven | 3.9+ | `mvn -version` |
| Node.js | 20+ | `node -v` |
| npm | bundled with Node | `npm -v` |
| Python | 3.11+ | `python3 --version` |
| Git | any recent | `git --version` |

**Optional** (only needed for the production-datasource and containerized paths later in this guide):

- PostgreSQL 16+ with the PostGIS extension available
- Docker + Docker Compose v2

> **Disk space**
>
> The AI service's dependencies include PyTorch. Budget roughly **8 GB of free disk** for its virtual environment the first time you install it.

## 3. Clone the repository

```
git clone https://github.com/maddy-bit/SIH_Project.git
cd SIH_Project
```

The repository's default branch is `main` and contains everything described in this guide — there is nothing on any other branch you need.

## 4. Run it locally — all three services

Open three terminals. Each service runs independently.

### 1 Backend (Spring Boot)

```
cd backend
mvn spring-boot:run
```

Leave this running. It starts on port 8080, seeds the demo camera network from `data/cameras.json`, and loads the mock vehicle registry from `data/vehicles.json`. It uses an embedded H2 database by default — zero extra setup required.

**Verify:**

```
curl http://localhost:8080/api/health
# {"status":"ok","service":"city-ai-backend","version":"0.1.0",...}
```

### 2 AI service (Python)

```
cd ai-service
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

This installs Ultralytics (YOLOv8), EasyOCR, OpenCV, and their dependencies. You do not need to run this as a standalone server for the next step — the ingestion script imports it directly in-process. It can also be run as its own HTTP service if needed:

```
.venv/bin/uvicorn main:app --port 8000 --reload
```

### 3 Frontend (Next.js)

```
cd frontend
npm install
npm run dev
```

Open **http://localhost:3000**. You should see a dark command-center dashboard with **SYSTEM ONLINE** in the header — confirming the frontend reached the backend. Vehicle search will return "no vehicle found" until you complete the next step.

## 5. Load the demo data

> **The database starts empty on every fresh clone.**
>
> The demo video clips are committed to the repo ( `videos/` ), but the AI-generated detections that come from processing them are git-ignored — they're generated data, not source. You must run the ingestion pipeline once locally to populate the database.

From the repo root, with the backend still running, process the four cameras that make up the primary demo scenario (target vehicle `MH12AB1234` ):

```
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-01.mp4 --camera CAM-01 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-03.mp4 --camera CAM-03 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-07.mp4 --camera CAM-07 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-11.mp4 --camera CAM-11 --start-t
```

Each command takes a few seconds — it loads YOLOv8 + EasyOCR, runs real detection and OCR on the video frames, and ends with `Stored 5/5 detections via http://localhost:8080/api/detections/ingest`. This is genuine model inference on real video frames, not fixture data.

**Optional — the full 11-camera network** (adds decoy vehicles so the search demonstrably isn't hardcoded to one plate):

```
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-02.mp4 --camera CAM-02 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-04.mp4 --camera CAM-04 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-05.mp4 --camera CAM-05 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-06.mp4 --camera CAM-06 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-08.mp4 --camera CAM-08 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-09.mp4 --camera CAM-09 --start-t
ai-service/.venv/bin/python3 scripts/process_video.py --video videos/CAM-10.mp4 --camera CAM-10 --start-t
```

`CAM-12` intentionally has no video — it demonstrates the "camera offline" state in the UI.

# 6. Verify everything works

## In the browser

1. Go to **http://localhost:3000**. The search box is pre-filled with `MH12AB1234`. Click **SEARCH VEHICLE**.
2. You should see a vehicle intelligence panel (SUV, White, 4 cameras, ~11 km, ~33 min, ~89% trajectory confidence), a detection timeline (CAM-01 → CAM-03 → CAM-07 → CAM-11 with time gaps and match confidences), and an evidence panel with the actual detected frame.
3. Click ▶ **Replay Trajectory** to animate the route on the map.
4. Try the **Live** search mode to watch detections stream in over a WebSocket.
5. Switch to the **City Traffic Summary** tab for the analytics dashboard (real computed aggregates, not canned numbers).

> **Map appears blank?**
> That's almost always a network blocking the free CARTO tile CDN (common on some corporate networks/VPNs) — every other panel works independently of the basemap tiles loading.

## From the command line

```
./scripts/smoke_test.sh
```

Runs 17 checks against every API endpoint — status codes, real response content, auth enforcement on protected routes. All should pass.

```
cd backend && mvn test
```

Runs the backend's own unit test suite (13 tests: plate normalization, geo-distance/bearing math, trajectory transition scoring).

# 7. Switch to the production database (PostgreSQL + PostGIS)

H2 is the zero-config default for local development. A real `postgres` Spring profile also exists — built and tested against an actual PostgreSQL 16 + PostGIS 3.4 database, including a genuine spatial query endpoint.

### 1 Create the database and enable PostGIS

```
sudo -u postgres psql -c "CREATE USER cityai WITH PASSWORD 'cityai_dev_password';"
sudo -u postgres psql -c "CREATE DATABASE cityai OWNER cityai;"
sudo -u postgres psql -d cityai -c "CREATE EXTENSION IF NOT EXISTS postgis;"
```

### 2 Run the backend against it

```
cd backend
SPRING_PROFILES_ACTIVE=postgres mvn spring-boot:run
```

On startup this automatically adds a generated `geography(Point,4326)` column and a GiST spatial index to the `cameras` and `vehicle_detections` tables, without touching any existing business logic. It powers a real spatial query:

```
curl "http://localhost:8080/api/cameras/nearby?lat=18.5308&lon=73.8474&radiusKm=3"
# returns CAM-01, CAM-02, CAM-03 — nearest first (real ST_DWithin query)
```

Then repeat the ingestion commands from Section 5 against this fresh database.

# 8. Run the whole stack with Docker Compose

```
docker compose up --build
```

This brings up PostGIS, the backend (on the `postgres` profile), the AI service, and the frontend together, wired via `docker-compose.yml` at the repo root. Each service also has its own multi-stage `Dockerfile` for a lean production image (the frontend uses Next.js's `output: "standalone"` mode).

> **Verify this yourself before relying on it in production.**
>
> These Dockerfiles were written to standard, well-established patterns, but the environment they were built in hit Docker Hub's anonymous image-pull rate limit before a full `docker build` could be completed and verified there. Run `docker compose up --build` on your own machine first — if anything needs a small fix, it should be a quick one, but don't assume it's untested-and-fine.

# 9. Go live — deploying to a real server

There is no existing hosting/CD pipeline in this repository — deploying to a live server is a manual step you (or your infrastructure) perform. Two practical approaches, from simplest to most scalable:

## Option A — Single VPS with Docker Compose (simplest)

Good for a demo deployment or small-scale production. Any VPS with Docker installed (DigitalOcean, AWS EC2, Hetzner, Linode, etc.), 2+ vCPUs and 4+ GB RAM as a starting point (the AI service is the heaviest consumer).

1. Provision a VPS, install Docker + Docker Compose.
2. `git clone` this repository onto the server.
3. Create a `.env` file (or export env vars) overriding the insecure local defaults — see Section 10.
4. Run `docker compose up --build -d` to start everything in the background.
5. Put a reverse proxy in front of it — **Nginx** or **Caddy** — routing:
   - `/` → frontend container, port 3000
   - `/api/*` and `/ws/*` and `/evidence/*` → backend container, port 8080
6. Terminate HTTPS at the reverse proxy with a free certificate via **Let's Encrypt** (Caddy does this automatically; with Nginx, use `certbot`).
7. Point the frontend's `NEXT_PUBLIC_API_BASE_URL` build argument at your public domain (e.g. `https://your-domain.com`) instead of `localhost:8080`, and rebuild the frontend image.

```
# Example minimal Caddy config (Caddyfile) for HTTPS + routing:
your-domain.com {
    handle /api/* {
        reverse_proxy backend:8080
    }
    handle /ws/* {
        reverse_proxy backend:8080
    }
    handle /evidence/* {
        reverse_proxy backend:8080
    }
    handle {
        reverse_proxy frontend:3000
    }
}
```

## Option B — Split managed hosting (more scalable)

| Service | Where it fits well |
|---|---|
| Frontend (Next.js) | Vercel, Netlify, or any static/edge host — set `NEXT_PUBLIC_API_BASE_URL` to your backend's public URL at build time |
| Backend (Spring Boot) | Render, Railway, Fly.io, AWS Elastic Beanstalk, or a container on any cloud — needs the `postgres` profile pointed at a managed database |
| Database | A managed PostgreSQL with the PostGIS extension enabled — AWS RDS, Render Postgres, Supabase, Neon, Crunchy Data all support PostGIS |
| AI service | A host with enough CPU (or GPU, for real scale) — a separate container/VM; not typically serverless-friendly given model load time |

Whichever option you choose, the environment variables that need setting in a live deployment are the same ones documented for local Docker use:

| Variable | Purpose | Local default (change in production) |
|---|---|---|
| `DB_URL`, `DB_USERNAME`, `DB_PASSWORD` | PostgreSQL connection | `jdbc:postgresql://localhost:5432/cityai` / `cityai` / `cityai_dev_password` |
| `CORS_ALLOWED_ORIGINS` | Which frontend origin(s) may call the API | `http://localhost:3000` |
| `INGEST_USERNAME`, `INGEST_PASSWORD` | HTTP Basic credentials for the detection-ingest and audit-log endpoints | `ingest` / `change-me-in-production` |
| `AI_SERVICE_URL` | Backend's reference to the AI service (used by future direct-call phases) | `http://localhost:8000` |
| `NEXT_PUBLIC_API_BASE_URL` | Frontend's reference to the backend — baked in at build time | `http://localhost:8080` |
| `SPRING_PROFILES_ACTIVE` | Set to `postgres` to use the real datasource instead of H2 | unset (H2) |

## 10. Security checklist before going live

This project is an honestly-scoped prototype (see Section 13) — it is **not** hardened for production exposure out of the box. Before putting it on the public internet, at minimum:

- **Change** `INGEST_USERNAME` / `INGEST_PASSWORD` — the committed defaults are for local development only.
- **Change** `DB_PASSWORD` to a strong, unique value.
- **Terminate HTTPS** at your reverse proxy — never serve this API or frontend over plain HTTP in production.
- **Tighten** `CORS_ALLOWED_ORIGINS` to your exact production frontend domain — never `*`.
- The investigation dashboard's **read endpoints are currently unauthenticated** by design (see the README's production gap table) — do not expose this publicly without adding a real login layer first if the data is sensitive.
- Do not enable the H2 web console (`spring.h2.console.enabled`) in any environment reachable from outside your machine.
- Rotate any credentials that were ever committed to version control history, even as "dev defaults."

# 11. API reference

| Endpoint | Purpose |
|---|---|
| `GET /api/health` | Liveness check |
| `GET /api/cameras` | List all cameras |
| `GET /api/cameras/{id}` | One camera |
| `GET /api/cameras/nearby?lat=&lon=&radiusKm=` | Real PostGIS spatial query (postgres profile only — 501 on H2) |
| `GET /api/vehicles/{plate}` | Vehicle intelligence summary |
| `GET /api/vehicles/{plate}/detections` | Raw detection events, chronological |
| `GET /api/vehicles/{plate}/trajectory` | Reconstructed route with per-transition confidence |
| `POST /api/detections/ingest` **AUTH** | Store one detection event |
| `POST /api/detections/ingest/batch` **AUTH** | Same, batched |
| `GET /api/analytics/overview` | City-wide computed aggregates |
| `GET /api/audit/recent?limit=` **AUTH** | Who searched what, when |
| `GET /ws/live/{plate}` (WebSocket) | Live-simulation detection stream |
| `GET /evidence/{cameraId}/{file}` | Saved evidence frame image |

**AUTH** = requires HTTP Basic auth.

# 12. Troubleshooting

| Problem | Fix |
| --- | --- |
| `video not found` from `process_video.py` | Run it from the repo root, not from inside `scripts/` or `ai-service/`. |
| Port already in use (8080 / 3000 / 8000) | Stop the other process, or override with `SERVER_PORT` (backend) / `-p` flag (frontend). |
| First AI-service run is slow | YOLOv8 weights (~6 MB) and EasyOCR models (~50–100 MB) download on first use only. Later runs are fast. |
| `mvn` can't find Java 21 | Check `JAVA_HOME` points at a JDK 21 install. |
| Map tiles don't load | Your network is likely blocking the free CARTO CDN — every other UI panel is unaffected. |
| `docker compose up` fails to pull images | Could be a registry rate limit on your network — retry, or authenticate `docker login` first. |
| 401 on `/api/detections/ingest` | Expected without credentials — pass `-u ingest:<password>` or use `scripts/process_video.py`, which sends them automatically. |

# 13. What's production-ready vs. not

Condensed from the full gap analysis in the repository's `README.md`:

| Area | State |
| --- | --- |
| Core AI pipeline (detection, tracking, OCR) | DONE Real pretrained models, tested on real footage |
| Trajectory reconstruction & confidence scoring | DONE Real, reasoned scoring — not just timestamp ordering |
| Investigation UI, live mode, analytics | DONE Complete, built on real data |
| Database | DONE Real PostgreSQL + PostGIS profile, tested end-to-end |
| Auth | PARTIAL Writes and audit log gated; UI reads still open |
| Audit logging | DONE Every search/trajectory view recorded |
| CI | DONE Confirmed passing on GitHub Actions |
| Docker | PARTIAL Written, not build-verified in the dev sandbox — verify locally |
| Full login / RBAC | TODO Not built — natural next step |
| Real vehicle registry / RTSP ingestion / GPU serving | TODO Needs external access/infra this prototype can't obtain on its own |

For the full detail — including a prioritized roadmap of what to build next, in order — see the **"What this is (and isn't)"** section of `README.md` in the repository.

> **Questions or issues?**
> Open an issue on the repository, or check `GETTING_STARTED.md` for the shorter, quick-start version of this same guide.